Lab 2: Epics and User Stories

Feature Flag & Dynamic Config Manager (Problem Statement #43)

Gnanasagar H | PES1UG24CS171 | Section 5 - 'C'

The five functional requirements from Lab 1 became five epics in Jira. Each epic holds the user stories needed to meet its requirement, and the two non-functional requirements are covered by stories inside the epics they constrain (NFR-001 under flag evaluation, NFR-002 under SDK sync). Story points use the Fibonacci scale (1, 2, 3, 5, 8). Work items were split across two sprints on the Jira Scrum board.

Note on work types: this team-managed Jira project only offers Epic, Task and Subtask, so each user story was created as a Task, written in the As a / I want / So that format and labelled user-story.

Epics

Key	Epic	Source	Priority	Description	Points
SCRUM-5	Flag Evaluation & User Targeting	FR-001, NFR-001	High	Evaluate targeting rules (User ID hash bucket, Beta cohort) and return the correct on/off value of a flag for each user, quickly and only to authenticated callers.	16
SCRUM-6	Percentage Rollout Management	FR-002	High	Let a Release Manager roll a flag out to a percentage of users (0 to 100, 1% steps) per environment, so a bad release reaches only a small group first.	9
SCRUM-7	Environment Overrides	FR-003	High	Give a flag separate values in development, staging and production, so a feature can be tested in staging without reaching real users.	10
SCRUM-8	Real-Time SDK Flag Sync	FR-004, NFR-002	Medium	Push saved changes to every connected SDK within 5 seconds, and keep SDKs serving cached values if the service goes down.	19
SCRUM-9	Audit Log & Rollback	FR-005	Medium	Record who changed what, when and where, and let a Release Manager restore any earlier version of a flag.	10

SCRUM-5: Flag Evaluation & User Targeting (FR-001, NFR-001)

Evaluate targeting rules (User ID hash bucket, Beta cohort) and return the correct on/off value of a flag for each user, quickly and only to authenticated callers.

Key	User story	Priority	Points	Sprint
SCRUM-10	As a client SDK, I want a flag evaluated against the user's ID hash bucket, so that the same user always gets the same result.	High	5	1
SCRUM-11	As a Release Manager, I want to add a Beta cohort targeting rule to a flag, so that opted-in users see the feature before everyone else.	High	3	1
SCRUM-12	As a host app developer, I want flag evaluation to answer in under 15 ms (p99), so that it never slows down my request path.	High	5	2
SCRUM-13	As a security owner, I want only authenticated SDK requests over TLS 1.2+ accepted, so that unreleased flag data cannot leak.	High	3	1

SCRUM-6: Percentage Rollout Management (FR-002)

Let a Release Manager roll a flag out to a percentage of users (0 to 100, 1% steps) per environment, so a bad release reaches only a small group first.

Key	User story	Priority	Points	Sprint
SCRUM-14	As a Release Manager, I want to set a rollout percentage per environment from the dashboard, so that I can release gradually.	High	3	1

SCRUM-15	As a Release Manager, I want rollout values outside 0 to 100 rejected, so that a typo cannot break a release.	Medium	1	1
SCRUM-16	As a QA engineer, I want a 10,000-user distribution test, so that I can prove a 25% rollout reaches about 25% of users.	High	5	1 -> 2

SCRUM-7: Environment Overrides (FR-003)

Give a flag separate values in development, staging and production, so a feature can be tested in staging without reaching real users.

Key	User story	Priority	Points	Sprint
SCRUM-17	As a Software Engineer, I want a flag to hold separate values in dev, staging and production, so that I can test in staging without touching production.	High	5	1
SCRUM-18	As a client SDK, I want an environment-scoped SDK key, so that I only ever read my own environment's values.	High	3	1
SCRUM-19	As a Software Engineer, I want to compare a flag's values across environments side by side, so that I can spot drift before a release.	Low	2	2 -> backlog

SCRUM-8: Real-Time SDK Flag Sync (FR-004, NFR-002)

Push saved changes to every connected SDK within 5 seconds, and keep SDKs serving cached values if the service goes down.

Key	User story	Priority	Points	Sprint
SCRUM-20	As a Release Manager, I want saved changes pushed to every connected SDK within 5 s, so that I can switch a broken feature off without a redeploy.	Medium	8	2
SCRUM-21	As a host app, I want the SDK to keep serving cached values when the service is unreachable, so that my app keeps working during an outage.	High	5	2
SCRUM-22	As a Release Manager, I want a warning listing SDKs that did not confirm an update, so that I know which clients are out of sync.	Medium	3	2
SCRUM-23	As a QA engineer, I want a 1,000-client sync load test, so that I can verify the 5-second limit.	Medium	3	2

SCRUM-9: Audit Log & Rollback (FR-005)

Record who changed what, when and where, and let a Release Manager restore any earlier version of a flag.

Key	User story	Priority	Points	Sprint
SCRUM-24	As a Release Manager, I want every change logged with user, time, environment, old and new value, so that I can trace what happened during an incident.	Medium	3	2
SCRUM-25	As a Release Manager, I want to filter the audit log by flag, user and date, so that I can find a specific change quickly.	Low	2	2
SCRUM-26	As a Release Manager, I want to roll a flag back to any earlier version, so that I can undo a bad change fast.	Medium	5	2

Setup tasks (no epic)

Key	Task	Priority	Points	Sprint
SCRUM-1	Set up service repository and CI pipeline	2	High	1
SCRUM-2	Design the flag, environment and version data schema (sub-task SCRUM-4: ER diagram)	3	High	1
SCRUM-3	Provision development, staging and production environments	2	Medium	1

Estimation summary

Total backlog: 71 story points across 17 user stories and 3 setup tasks. Sprint 1 was planned at 35 points and Sprint 2 at 36 points. The 8-point story (SCRUM-20, real-time push) is the largest because it involves a new service, a

WebSocket protocol and delivery confirmation, all with some uncertainty. The 1-point story (SCRUM-15, input validation) is a single, well-understood check.

Backlog screenshot

